

HAVEN: A Hybrid Anomaly Detection System for Intra-Vehicular CAN-Bus Communication Using Rule-Based and Neural Networks

Shaurya Purohit^{ID} and Manimaran Govindarasu^{ID}, *Fellow, IEEE*

Abstract—Modern vehicles rely on ECUs connected over an Intra-Vehicular CAN-bus network to facilitate data exchange. However, in recent times, the quantity of potential attack surfaces for malicious cyberattacks, such as DOS, Fuzzy, etc., has notably surged owing to their amplified connectivity. Unfortunately, the CAN bus solitarily cannot ensure its protection due to its deficiency in security components, leading to grave safety and security issues like disabling the brakes, etc., and hence requires a reliable mechanism for detecting anomalies in the system. To address these challenges, this paper proposes HAVEN, a Hybrid Anomaly Detection System for Intra-Vehicular CAN-bus Communication using combinations of Rule-based with Machine Learning techniques and Neural Networks (Binary and Multiclass Classification forms). The experimental results show that our proposed hybrid models achieve high detection accuracy on different datasets incurring significantly low execution time and high F1-score. This is attributable to the parallel execution and multi-threading nature of the Machine Learning and Neural Networks employed in conjunction with the Rule-based techniques. Upon comparing the results of both models, our evaluation demonstrates that the second model incorporating neural networks yields superior results, establishing its potential as a highly efficient and promising solution and opening future avenues for further research work.

Index Terms—CAN-bus, anomaly detection, machine learning, neural networks, multiclass classification.

I. INTRODUCTION

THE transportation sector is undergoing significant transformation, propelled by rapid advancements in autonomous and connected vehicle technologies. Modern automated and connected vehicles have intricate designs that merge different automated driving functions, and diverse communication interfaces [1].

The inter-connectivity and autonomy of vehicles are increasing as Electronic Control Units (ECUs) communicate with each other to manage various functions, including engine control, telematics control, and airbag deployment. Different communication standards have been established within vehicles. Nearly ubiquitous, Controller Area Network (CAN) act as intermediaries for data exchange between ECUs by furnishing a communal network with a standard protocol. It is a serial-bus communication protocol that allows sensors and actors to communicate

with ECUs. Additionally, CAN offers a reliable error detection mechanism that promotes uniform transmission and potentially diminishes wiring expenses, weight, and intricacy [2]. CAN operates as a peer-to-peer network, meaning there is no designated master that governs transmission. CAN nodes or devices initiate transmissions on the CAN frame only when the bus is unoccupied, and the node is prepared to transmit. The communication protocol used by the CAN-bus is designed to be a broadcast domain, which implies that any message sent is received by all the nodes linked to it. The CAN frame lacks the recipient's address but includes a unique message priority bit known as the Arbitration ID. Nodes receive the frame by matching it with an arbitrary bit of the sent frame, and each node decides whether or not to accept the received frame.

In situations where several nodes attempt to send messages simultaneously, they employ a non-destructible system called Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA) and Arbitration on Message Priority (AMP) to contend for access to the bus. The arbitration process in this system prioritizes messages with lower ID, allowing them to be transmitted first, while nodes with lower priority must wait until higher priority nodes complete their execution.

In light of their progressively getting connected, these modern vehicles have emerged as a novel arena for hackers to exploit, thereby making automotive cybersecurity a matter of profound importance within the present-day automotive industry. In fact, every single node within these vehicles presents a potential point of target for serious cyber-attacks. Given these risks, developing and implementing a rigorous and comprehensive methodology for identifying and addressing critical security threats has become indispensable. Eclectic techniques, such as message authentication and sender identity, have been proposed in previous studies to ensure the legitimacy and reliability of CAN messages [3], [4], [5], [6], [7]. These methods are efficacious in preventing unauthorized access to the CAN network but are limited in their ability to defend against attacks emanating from compromised ECUs. Since ECUs are a vital part of the vehicular network and are typically equipped with the necessary credentials for secure communication, they can become a source of security threats, hence allowing these malicious actors to possess the capability to infiltrate the vehicular networks and transmit unauthorized CAN messages that disrupt the regular functioning of the vehicle, thereby leading to consequential safety repercussions.

For instance, consider the dire consequences that could arise if a connected vehicle carrying numerous passengers were to

Received 11 October 2023; revised 22 January 2026; accepted 8 March 2026.
Date of publication 11 March 2026; date of current version 12 May 2026.
(Corresponding author: Shaurya Purohit.)

The authors are with Iowa State University, Ames, IA 50011 USA (e-mail: shaurya1@iastate.edu; gmani@iastate.edu).

Digital Object Identifier 10.1109/TDSC.2026.3673151

experience a malicious attack on its steering control node [8]. Additionally, an attacker with malicious intent could target a specific frame of the CAN-bus, causing the throttle position to be compromised and rendering the vehicle immobile. While not necessarily life-threatening, an adversary motivated by monetary gain could exploit vulnerabilities in the car's entertainment system through CAN-bus, forcing the vehicle to come to a halt and displaying ransom messages on the entertainment system screen and thereby making the owner compelled to pay the ransom demanded to regain control of the car.

Given the lack of encryption and authentication procedures, we hold the view that an Anomaly Detection System (ADS) would be an efficacious method for detecting anomalous data and malevolent attacks through network traffic data transmitted between the vehicle and other devices. If an attacker executes an attack by injecting harmful traffic into the CAN-bus, the ADS sifts through the traffic and notifies the user of any unusual activity by contrasting the normal and aberrant behaviors.

While ADSs have been widely studied here, many prior approaches fall short in key areas. Most focus on a limited range of attacks—often ignoring threats like spoofing or impersonation—and rely on simulated data, reducing real-world applicability. Additionally, several models emphasize accuracy but neglect execution time, which is critical for responsiveness in CAN-bus environments. Many also lack lightweight preprocessing, making them inefficient for resource-constrained systems.

Building on these prior efforts, many ADSs have also attempted to leverage distinct features of CAN messages. However, their predicament seems to be a deficiency of suitable and potent detection and mitigation solutions for threats and attacks. Several of these systems also exploit the recurring and periodic nature of CAN traffic [9], [10]. While they prove to be successful in creating efficient ADS, they falter when it comes to identifying with non-periodic attack messages, which poses a significant challenge. To determine whether changes to packets have occurred, other researchers have researched on distinct metrics as well. For instance, one of the algorithms assesses modifications in packet entropy for every bit of the CAN ID to establish if an injection attack has taken place and obstructs the message. Another intrusion detection algorithm estimates the Hamming distance of attack payloads and juxtaposes it with Hamming distance data of reference to detect attacks [11]. The majority of the suggested ADS lack efficacy due to their high incidence of false positive and false negative alerts.

To overcome these limitations, this work proposes HAVEN, novel two-step Hybrid ADS's for Intra-Vehicular CAN-bus Communication (refers to the implementation of the CAN bus protocol for communication between ECUs within a vehicle, which is a subset of the broader in-vehicle networks) that maintain an equilibrium between detection rates and efficiency. The initial model employs rule-based detection coupled with tree-based ML algorithms, such as Decision Tree (DT), Random Forest (RF), Extreme Gradient Boosting (XGBoost), Categorical Boosting (CatBoost), and Light Gradient Boosting Machine (LightGBM). The second model employs rule-based discovery in combination with deep neural networks, including Binary and Multiclass classifiers. By integrating rule-based detection with machine learning and neural networks, our framework ensures high detection accuracy and low false negatives while effectively

addressing the challenge of complex non-periodic attacks (e.g., Impersonation, Spoofing) that lack consistent timing or structure and often bypass traditional frequency- or sequence-based methods. The rule-based layer serves as a lightweight, rapid filter that flags overt timing and sequence anomalies in CAN traffic. The subsequent ML or neural network layer—depending on the model—focuses on learning more subtle and irregular patterns that evade deterministic rules. This two-step design allows us to efficiently detect both frequent and complex attack behaviors with minimal execution time.

The principal contributions put forth by the medium of this paper are:

- *HAVEN*: Development of two novel hybrid ADS models comprising two distinct stages, which enables the attainment of high detection accuracy incurring low latency. The first stage employs resilient and efficient rules to minimize the burden on the resource-intensive second stage. In turn, the second stage harnesses advanced machine learning and neural network techniques to effectively detect intricate attacks that can elude the first stage. These hybrid ADS frameworks have the capability to identify a wide range of attacks that may pose a significant challenge for individual detection approaches to detect accurately.
- *Enhancing Classification*: Furthermore, the second model, which amalgamates rule-based and neural network techniques, is trained to cater to both binary and multiclass classification scenarios to enhance the current state of the art, an aspect that was overlooked in earlier studies.
- *Expanded Assessment*: The efficacy of the proposed models are gauged using three large CAN-bus datasets that are widely used and publicly available (CAN Intrusion dataset, Car Hacking dataset, and Automotive CAN-bus Intrusion dataset). The results show that the proposed framework outperformed prior studies in terms of detection accuracy and execution time (latency).
- *Comprehensive Attack model*: The proposed models underwent testing on actual car-hacking data, encompassing a diverse array of attacks such as DoS, Fuzzy, Impersonation, Spoofing and Replay attacks. The attack model utilized was more realistic and encompassing, covering a majority of the well-known attack types in the literature, in contrast to prior studies that only focused on specific attacks. The efficacy of the proposed models are demonstrated through the calculation of various evaluation metrics.

The subsequent sections of this paper are structured as follows: Section II presents the Problem Formulation. Section III elucidates the related research work. Section IV expounds the Rule and Machine Learning based Hybrid ADS model. Section V details the insights into the implementation and evaluation of the Rule and Neural Networks based Hybrid ADS model. Finally, Section VI concludes the paper.

II. PROBLEM FORMULATION

A. CAN Bus

CAN is a bus communication protocol designed to broadcast messages, with the ability to transmit frames that can contain up to 64 bits of data. In Fig. 1, the standard format of a CAN

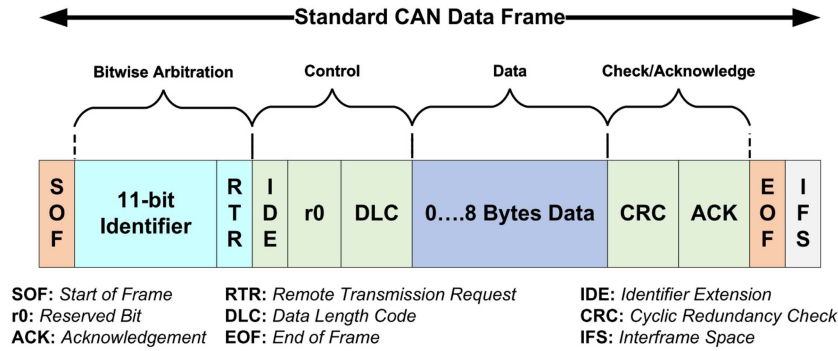


Fig. 1. CAN Message Frame.

frame is presented. The CAN frame includes various fields such as the Start of Frame (SOF), Control Field, Arbitration Field (CAN ID), Data Field (payload), CRC Field, Acknowledge Field (ACK), and End of Frame (EOF).

The Start Frame, which has a chief bit of '0', marks the beginning of a CAN message. The CAN ID field is capable of supporting different identifier formats, such as a base frame of 11 bits, an extended frame of 29 bits, and an 18-bit extension. The Remote Transmission Request field determines whether a message is being transmitted or received. The Control Field specifies the transmitted data length, which ranges from 0 to 8 bytes. The Data Field contains the actual content and critical information sent to the network's intended node. The Cyclic Redundancy Check is a partial check used to detect any alterations made to the original data. The Acknowledge Field is used to validate that the node has received the data correctly. The End of the Frame marks the message's conclusion.

CAN is message-oriented and does not require sender and receiver addresses. Instead, each CAN frame has a predetermined ID and message structure outlined in the DBC file. The DBC file is a confidential file in an exclusive format, such as a database, that is part of a specific vehicle configuration. During compilation, the ECU is set to accept only CAN messages with a specific CAN ID and disregard all other messages.

CAN ID refers to the designated ECU's destination address, and its payload has a maximum length of 8 bytes with no encryption or authentication mechanisms. As a result of this, CAN is susceptible to message injection attacks due to its broadcast communication strategy and the lack of authentication mechanisms. Adversaries can leverage this vulnerability to launch DoS and Spoofing attacks on the CAN-bus, thereby occupying resources and extracting sensitive information such as RPM and Gear information. Furthermore, Replay and Fuzzy attacks represent another class of frequently encountered categories of attacks that target the CAN-bus. These attacks occur when the attacker inserts a message into the bus to drive it into, signaling an abnormal state or malfunction, thereby providing a false impression to the vehicle's control systems.

Many studies have posited the notion that an adversary can acquire absolute authority by gaining entry to the CAN network through a plethora of attack interfaces, including but not limited to TPMS, Bluetooth, telematics, and OBD2. The deficiency of the fundamental security protocols renders the CAN-bus system susceptible to a variety of severe assaults. An adversary could

easily execute an array of harmful actions, including but not limited to injecting harmful messages to observe network traffic or launching hostile attacks, while nodes within the network are obliged to process the message automatically without identifying its source.

B. Attack Model

As stated earlier, attackers are extremely incentivized to exploit the vulnerabilities present in the CAN Bus. In this paper, we worked with some of the primary categories of in-vehicle attacks that can be executed on the CAN-bus system: Denial of Service (DOS), Fuzzy, Impersonation, Spoofing, and Replay attacks. Fig. 2 portrays the prevalent categories of attacks on the CAN-bus that have been addressed in our work.

Denial of Service Attack: DOS attacks on the CAN-bus system involve sending many requests to the targeted nodes, which consumes their resources and may lead to vehicle unavailability or mishaps [12]. As Autonomous Vehicles (AVs) use wireless technology, DOS attacks can be easily executed by adversaries. In this type of attack, the attacker inundates the bus with messages having high frequency and high priority (such as messages with a CAN ID of 0x000 or some different ID that is smaller than all valid message IDs) to congest the bus and occupy its entire bandwidth. This causes the true messages to be unable to transmit successfully, eventually compromising the vehicle. If the attackers gain access to the vehicle, they can easily insert flooding assaults into the CAN-bus network, thereby interrupting or disabling the services of the ECUs in the vehicle.

Fuzzy Attack: The adversary launches an attack by injecting CAN messages with unpredictable CAN IDs, which results in a delay in the delivery of certain legitimate messages with higher priority (i.e., larger CAN IDs). This form of attack is more surreptitious than a DoS attack, owing to the random nature of the CAN IDs assigned to the injected messages, and a single malicious message injected of such nature into the CAN-bus network system can potentially result in the failure of the entire network, leading to unintended consequences such as the steering wheel shaking, automatic gear shifts, turning on/off indicator lights, and other similar effects.

Impersonation Attack: In this type of attack, the attacker deliberately injects modified CAN messages with specific CAN IDs to cause confusion within the ECU. If the attacker's CAN IDs are indistinguishable from legitimate ones, it is challenging

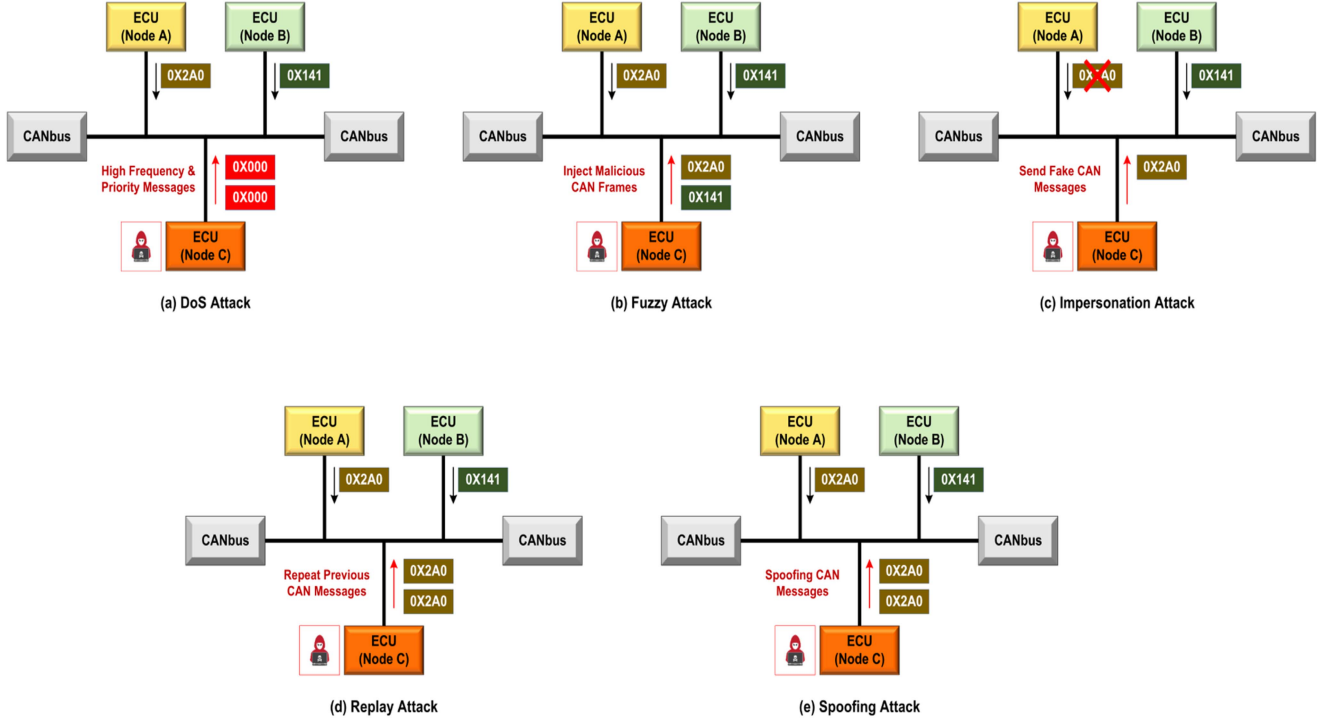


Fig. 2. Different Attack Scenarios.

for the ECU to differentiate between genuine and malicious messages. Moreover, these attacks are non-periodic and can occur sporadically, adding another layer of complexity to the detection process. As a result, the ECU can become overwhelmed with incoming messages, potentially leading to system failure. For instance, an attacker could impersonate the anti-lock braking system (ABS) ECU's ID and send a message to disable the system, causing the brakes to malfunction.

Replay Attack: In a replay attack, the attacker intercepts, records valid messages sent over the bus, and then replays them to the same or other nodes on the bus later. This type of attack can be particularly dangerous in systems where messages are used for control safety-critical functions, such as in automotive control systems. The replayed message may cause unintended or dangerous behavior, leading to serious consequences such as accidents or equipment damage. For example, suppose a message with a specific CAN ID is sent to turn on a car's headlights. In that case, an attacker could intercept and record that message and later replay it multiple times, causing the headlights to repeatedly turn on and off, which could be dangerous for the driver and other vehicles on the road.

Spoofing Attack: A spoofing attack on a CAN-bus refers to an attack in which an attacker sends a message to the bus with a fake source address to deceive the recipient node into thinking that the message came from a trusted source. The attacker modifies the message's ID and data payload fields to make it appear legitimate. This type of attack can have serious consequences, such as compromising the integrity of the data, disrupting the system's normal functioning, or causing safety-critical systems to fail. For example, an attacker could spoof a message from the ECU to the braking system, instructing it to apply the brakes or release them suddenly, causing a potential accident. Similarly,

an attacker could spoof a message to the ECU, instructing it to shut down the engine or make other critical changes. Notably, spoofing attacks can exhibit a non-periodic nature, meaning they can occur at irregular intervals, making it challenging for conventional security measures to identify and mitigate them effectively. This type of attack can have serious safety implications, especially in autonomous vehicles where the system relies heavily on the information being communicated via the CAN-bus.

C. Problem Statement

On a vehicle's CAN-bus network, the complexities and diversity of network traffic, encompassing both benign and malicious activities, pose challenges in accurately identifying malevolent attacks such as DOS, Fuzzy, Impersonation, Replay, and Spoofing attacks. Achieving high precision and minimizing false negatives in detection are critical objectives. This paper addresses this problem by proposing two potent ADS models, designed using combinations of Rule-based, Machine Learning, and Neural Networks techniques, that exhibit efficient performance in enhancing the accuracy of identifying these attacks while ensuring low latency.

III. RELATED WORK

In recent times, a significant focus has been placed on autonomous vehicles, which have the potential to significantly impact people's daily lives. Researchers have devoted their attention to examining the features and roles of these vehicles in real-life scenarios to improve their functionality and enhance the safety infrastructure on roads. Consequently, the development

of IDS and ADS for AVs and connected vehicles has become increasingly paramount. In this section, we will discuss some of the previous studies on detecting network intrusions and anomalies in autonomous vehicle systems. Researchers have employed a diverse range of sophisticated methods, including statistical analysis, machine learning algorithms, frequency-based analysis, neural networks, and other techniques to detect anomalies in the CAN-bus. Barletta et al. [13] proposed an IDS for CAN using an X-Y fused Kohonen network and the k-means algorithm. The suggested technique, called XYF-K, has a high accuracy for detecting intrusions on the CAN-intrusion dataset, but suffers from high computational complexity. Schmidt et al. [14] developed a spline-based IDS using the knot flow classification (KFC) approach for vehicular networks and evaluated it on the NSL-KDD dataset. However, this model did not achieve low latency. Oladimeji et al. [15] proposed CANGuard, an IDS tailored for CAN-enabled vehicles to detect DoS attacks and identify the originating node in real time. The system was developed using a simulated CAN environment with four nodes representing different vehicle components. Various machine learning models were trained on a generated dataset exploiting CAN bus arbitration features. While the Multi-Layer Perceptron (MLP) model showed good detection capabilities, it underperformed with node identification, achieving less than 50% accuracy. Taylor et al. [16] introduced a method for detecting CAN-bus attacks that involves an anomaly detector utilizing a Long Short-Term Memory (LSTM) neural network. This detector functions by training the LSTM network to forecast the next data word sent from each sender on the bus. Any significantly unexpected bits in the actual next word are identified as anomalies and flagged accordingly. The research conducted by Li et al. [17] presents an innovative technique for identifying anomalous CAN-bus messages through the fusion of a deep neural network and a triplet loss network. Specifically, the DNN is first employed to train CAN-bus sequences and extract feature vectors which are then utilized for the triplet loss network, which calculates similarities between three randomly selected data sequences from the CAN-bus database. Ali Alheeti et al. [18] introduced an IDS using Artificial Neural Network (ANN) to oversee the car's status based on information gathered from the internal buses. The primary objective of this system is to ensure the security and safety of the internal network. To achieve this, the parameters of the ANN structure are established by training it with CAN packet information to derive the essential statistical properties of normal and attacking packets. This system aims to extract the relevant attribute to classify the attack and protect the car from any potential security threat. The research presented by Kosmanos et al. [19] presents a cross-layer Intrusion Detection System (IDS) utilizing probabilistic approaches and Machine Learning (ML) techniques, specifically KN and RF. Olufowobi et al. [20] suggested an anomaly-based IDS for CAN attack detection using the adaptive cumulative sum (CUSUM) technique. This technique can effectively identify intrusions based on statistical changes, but it may take a significant amount of time to execute. Song et al. [21] proposed a lightweight IDS for in-vehicle networks based on time interval analysis (TIA) of CAN communications. Another proposed approach [22] suggested using a combination of in-vehicle sensors to verify message formality, position, data range, data plausibility, and other

factors. However, the latter method lacks a functional implementation. In their study, Loukas et al. [23] developed an IDS using multiple machine learning classifiers for in-vehicle network systems. They conducted experiments by introducing threats into a cloud-based system and testing it with robotic cars. After achieving an overall accuracy of 86.9%, they concluded that the IDS is effective in detecting vehicular attacks. Alogailly et al. [24] proposed an IDS for the Internet of Vehicles (IoV) and connected cars using the deep belief network (DBN) and decision tree (DT) algorithms. Their approach achieved good accuracy on the NSL-KDD dataset. In a research paper [25], a clock-based IDS was proposed that detects anomalies and fingerprints Electronic Control Units (ECUs) by utilizing the periodicity of many CAN signals. However, this approach has limitations as it is not effective against attacks that involve aperiodic messages, despite its lightweight design. In [26], our previous work, we presented the preliminary rendition of our research endeavor, wherein we proposed the approach utilizing a combination of rule-based and machine-learning techniques. The proposed approach was solely trained and evaluated on the CAN-intrusion dataset. Compared to prior works, our HAVEN framework demonstrates clear advantages in both detection accuracy and operational efficiency. Unlike the majority of related approaches that either rely on simulated data or handle a limited number of attack types, HAVEN is evaluated using multiple public, real-world CAN datasets, making it more practical for real-world applications. Furthermore, our hybrid approach, which combines rule-based with ML and Neural Network techniques, offers faster execution times while maintaining high detection accuracy (around 95%), particularly for complex attack patterns. HAVEN also supports both binary and multi-class classification, providing flexibility in detecting a broader range of attack types.

Table I compares some of the related works.

IV. RULE AND MACHINE LEARNING BASED HYBRID ADS MODEL

This research endeavors to devise a pioneering ADS model for vehicular networks, which can detect and pinpoint any malicious conduct on the CAN system in real-time and draw attention to any aberrations from standard or malignant behaviors. Our proposed methodology comprises a two-stage ADS model aimed at attaining the desired objectives. A robust rule-based system is put into operation in the first stage, while for the second stage, a tree-based ML algorithm system is employed.

The high-level design of the ADS model is presented in Fig. 3. The proposed two-stage ADS model utilizes a Rule-based system at the initial stage to filter out most attacks and only provides efficient detection before allowing remaining messages to move forward. Rules are designed to have a low rate of false negatives. Messages that do not pass through the rules in the system are deemed as an attack, and an alert is extended. Subsequently, the remaining messages are directed towards the ML-based model (Model1) for further processing. The multi-threading nature and parallelism of the ML techniques render the overall model to have a significantly reduced computational time. The messages that satisfy both stages are then transmitted to the vehicular network via the CAN-bus.

TABLE I
COMPARISON WITH OF SOME OF THE RELEVANT WORK

Reference	Attack Vector / Dataset	Model Used	Accuracy (>90%)	Execution Time	Classifier
Barletta et al. [13]	CAN / KDD CUP-99	KSOM + K Means	✓	N/A	Multi-class
D. A. Schmidt et al. [14]	VANET / NSL-KDD	Knot Flow Classification	X	N/A	Binary
Oladimeji et al. [15]	CAN / Own data + Simulation	MLP	X	N/A	Binary
Taylor et al. [16]	CAN / Own data + Simulation	LSTM Networks	✓	N/A	Binary
Li et al. [17]	CAN / CANoe Simulation	DNN + Triplet	X	✓	Binary
Ali Alheeti et al. [18]	VANET / OCTANE Simulation	ANN	✓	N/A	Binary
Kosmanos et al. [19]	VANET / SUMO Simulation	RF, KNN	✓	N/A	Binary
Our research	CAN / CAN Intrusion Dataset, Car Hacking Dataset, Automotive CAN bus Intrusion Dataset	Rule-based + Machine Learning, Rule-based + Neural Networks	✓	✓	Binary, Multi-class

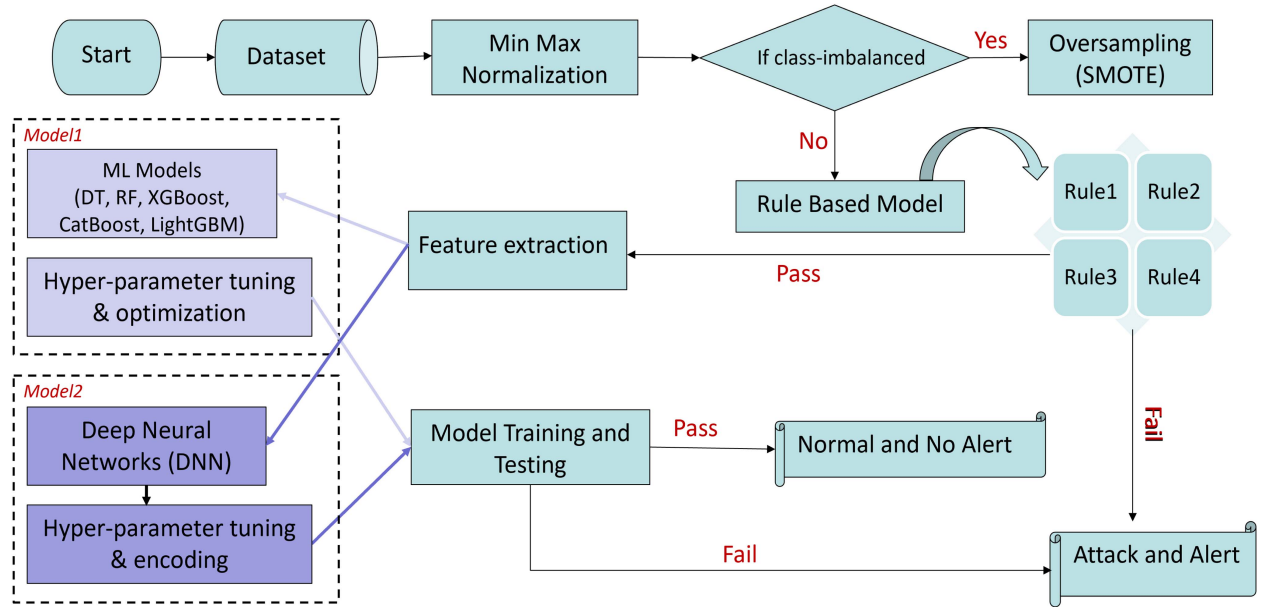


Fig. 3. Proposed Hybrid ADS Rule-based with Machine Learning and Neural Networks Model.

A. Data Pre-Processing

The foremost and preliminary measure to fabricate an ADS involves the accumulation of an adequate amount of network traffic data in both the normal and attack state. The CAN intrusion dataset is utilized for this function as it comprises data for both normal and attack data. Subsequently, the data undergoes pre-processing to render it more suitable for the development of ADS. The normalization of data often yields more optimal

results for training ML models. Consequently, the numerical values for each feature are constrained to a scale of 0.0 to 1.0, and the resulting normalized values, denoted as x_n , are expressed as:

$$x_n = \frac{x - \min}{\max - \min} \quad (1)$$

where x is the initial value, $\min$ and $\max$ are the minimum and maximum values of each feature.

TABLE II
DATA ATTRIBUTES OF CAN

Attributes	Explanation
Timestamp	Time when the data is recorded
DLC	Number of data bytes
CAN ID	Unique ID
Data [0-7]	Message value

Additionally, class imbalances are common in network data, where normal states dominate and attack-label instances are scarce. This can lead to low detection rates of anomalies. To address this issue, Synthetic Minority Oversampling Technique (SMOTE) [27] is utilized to generate additional data for the minority classes that lack sufficient data. SMOTE is a technique that examines the minority classes and creates new samples based on the K nearest neighbors.

Specifically, for each instance in the minority class, a random sample X_j is chosen from its k -nearest neighbors, and a new synthetic instance, X_n , can be expressed as:

$$X_n = X + \text{rand}(0, 1) * (X_j - X), j = 1, 2, \dots, k \quad (2)$$

where $\text{rand}(0, 1)$ denotes a random number between 0 and 1. The proposed system thus employs SMOTE to generate high-quality samples for pre-processing data.

Furthermore, the input data for the proposed system was initially in hexadecimal format. The conversion from hexadecimal to decimal format was necessary to meet the requirements of the machine learning model. The data was not decoded during this process. In cases where the data was missing, a value of -1 was used to replace the empty fields. In our work, ten different features and a CAN ID-based ADS were analyzed. The features included CAN ID, DLC, Data $[m_0 - m_7]$, and Time difference. CAN ID serves as a unique identifier for each CAN communication, while the DLC data bytes range from 0 to 8. The explanation of several attributes is shown in Table II.

B. Feature Extraction

To determine the importance of each feature, we use the information gain (IG) method, which calculates a score for each feature's relevance to the target variable. The IG approach is chosen for its low time complexity of $O(n)$, enabling it to quickly evaluate the significance of each feature. The relevance score of each feature is used to select the most relevant features for the task at hand. The IG value of using each feature, represented by the random variable X , is calculated based on the target variable T .

$$IG(T|X) = H(T) - H(T|X) \quad (3)$$

where $H(T|X)$ is the conditional entropy and $H(T)$ represents the entropy of target variable. If $IG(T|X) > IG(T|Y)$, a feature X is regarded more critical to the target T than feature Y .

Correlation analysis helps identify relationships between variables, allowing us to select the most relevant features that contribute to improving model performance [28]. Using similar approach, ten features were extracted from the identifier field and data field of CAN packets based on the CAN packet structure

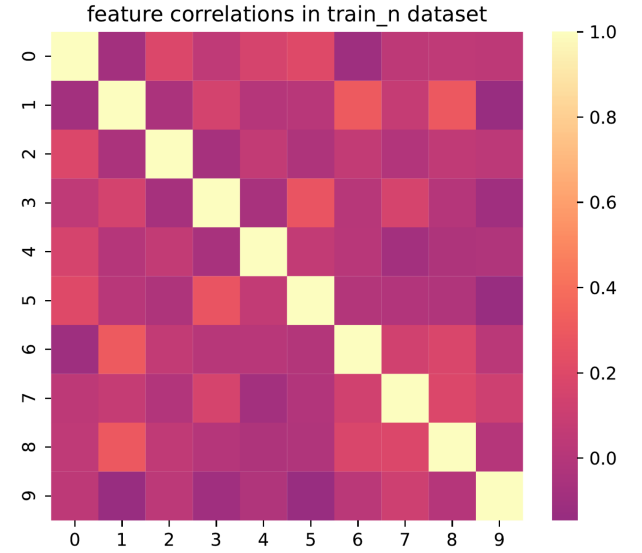


Fig. 4. Feature Correlation Map.

shown in Fig. 1. These features include CAN ID, Time difference, and DATA[0]-DATA[7], which were selected for ADS ML model development. The correlation between these features is depicted in Fig. 4.

C. Rule Based Intrusion Detection

The foremost stage of the proposed ADS model is a rule-based component, which offers efficient detection prior to moving on to the second stage. The characteristics of CAN traffic can be exploited to establish rule-based systems, and a plethora of rules can be implemented. The key objective is to determine a set of criteria that can filter out most of the attack messages in the first stage, thus preventing them from moving on to the time-consuming second stage. In order to achieve quick detection, it is crucial to select rules that possess a low false-negative rate and minimal processing latency.

Our model comprises of 4 rules which are based on the subsequent attributes:

- 1) *Valid ID*: A valid message transmitted via CAN protocol necessitates the presence of a singular and exclusive identifier. Any communication lacking such an identifier will be promptly recognized, identified as potentially malevolent, and subsequently flagged. Consequently, the message will be deemed invalid and omitted from the further stage.
- 2) *Message Frequency*: The rule underpinning this regulation stems from the observation that a vast majority of CAN packets are transmitted in a consistent and predictable pattern, thus leading to a recognizable frequency distribution.
- 3) *Message Sequence*: The underlying foundation for the sequence rule is the observation that a set of specific IDs inevitably accompanies a unique ID. For instance, this rule is particularly effective in detecting DOS attacks, as such attacks are often characterized by the presence of 0x0000 value in their messages, which can be promptly eyeballed through inspection.

- 4) *Time Interval*: The time interval rule is pertinent to CAN messages since they are primarily generated and transmitted periodically. This rule is particularly effective in identifying injected messages, as it scrutinizes the time intervals between incoming messages for any irregularities or anomalies.

D. ML Based Anomaly Detection

Decision tree, Random Forest, XGBoost, CatBoost and LightGBM are the ML techniques used in the ADS model.

- *Decision Tree*: The Decision Tree (DT) is a classification method that recursively splits data based on the attribute with the highest information gain. It builds a tree-like model of decisions and stops when reaching a predefined criterion, such as maximum depth or minimum samples per leaf node.
- *Random Forest*: Random Forest (RF) is an ensemble learning method that selects the final classification result based on majority voting from multiple decision trees. Each tree is trained on a random subset of data and features to increase diversity and prevent overfitting. The predictions from all trees in the forest are combined to make the final decision.
- *XGBoost*: XGBoost [29] is a boosting ML algorithm that combines decision trees using gradient descent, improving speed and accuracy. It iteratively trains trees on the residuals of the previous ones, correcting errors with each iteration to create a more accurate overall model.
- *CatBoost and LightGBM*: CatBoost [30] and LightGBM [31] are advanced gradient boosting algorithms. CatBoost excels in processing categorical data with minimal preprocessing, ideal for complex datasets. LightGBM reduces memory usage with histogram-based methods and leaf-wise growth, enhancing efficiency. Both algorithms like XGBoost, iteratively refine decision trees, offering precise corrections and notable accuracy on large datasets.

The rationales underlying the selection of these algorithms are:

- The primary motive for opting for these tree-based algorithms is their reduced computational time attributed to multi-threading utilization.
- Furthermore, the capability of these algorithms to deal with non-linear and high-dimensional data, which is characteristic of the network data being analyzed, also contributed to their selection.
- As most of these machine learning models are based on tree structures that incorporate ensemble learning, they tend to exhibit superior performance compared to single models such as KNN and SVM.
- Also, the ability to perform parallel execution is a notable advantage of these algorithms, resulting in a significant reduction in the time required for model training and refinement in detection accuracy.

Under the assumption that the number of instances is denoted by n , the number of features is represented by K , and the number of decision trees in ensemble models is represented by t , the time complexity of DT, RF, XGBoost, CatBoost and LightGBM algorithms can be expressed as follows:

- *DT* has a time complexity of $O(n^2 K)$.
- *RF* has a time complexity of $O(n^2 \sqrt{K} t)$.
- *XGBoost*, *CatBoost* have a time complexity of $O(n K t)$.
- *LightGBM* has a time complexity of $O(n \log n \cdot K t)$.

It is worth mentioning that the optimization of some hyperparameters is necessary to enhance the performance of the algorithms. Furthermore, the number of decision trees T represents a crucial parameter that greatly influences the performance of tree algorithms, as the majority vote of multiple decision trees influences their outcomes.

Consequently, the grid search approach is employed to determine the optimal parameter values. The models are initially trained using a small number of decision trees and a low-depth level to ensure accuracy. Subsequently, these two values are incrementally increased while checking for accuracy until overfitting is detected, which is characterized by a deterioration in the accuracy of the model.

E. Performance Evaluation

1) *Validation Metrics*: In order to assess the generalizability of the proposed model and prevent overfitting, both cross-validation and hold-out approaches are utilized. The train-test-validation split and model evaluation process employed for the dataset are as follows:

- 1) The dataset is split into training and test sets using a 70%-30% train-test split. Specifically, 70% of the data samples (chronologically ordered by timestamp) are assigned to the training set, while the remaining 30% (which follow the training portion) are reserved as the test set for final hold-out validation. This preserves temporal order and avoids potential data leakage by ensuring that no future data is used during model training.
- 2) To assess the performance of the proposed model across multiple areas of the dataset, 10-fold cross-validation is conducted on the training set. Specifically, in each iteration/fold of the 10-fold cross-validation, 90% of the original training set is used for model training, while the remaining 10% of the original training set is utilized as the validation set for model testing. This process is repeated ten times, with each subset serving as the validation set once, and the overall performance is evaluated as the average of the performance across all ten iterations/folds.
- 3) The model is tested on the untouched test set to evaluate the performance of the trained model obtained from Step 2. This step ensures that the model can generalize well to new, unseen data and provides an accurate measure of its performance.

The proposed ADS's model performance is evaluated using a range of metrics, including Accuracy, Precision, Recall(Detection Rate), and F1-Score. These metrics are computed using the values of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) determined by the proposed model. The following equations are used to calculate the metrics:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

TABLE III
SIZE OF THE DATASET

Attack Type	Number of Instances
Dos Attack	656,579
Fuzzy Attack	591,990
Impersonation Attack	995,472
Attack free state	2,369,868

$$F1 = \frac{2 * TP}{2 * TP + FP + FN} \quad (5)$$

$$Precision = \frac{TP}{TP + FP} \quad (6)$$

$$Recall = \frac{TP}{TP + FN} \quad (7)$$

TP refer to correctly identified anomalies, while TN are correctly identified normal instances. FP occur when normal data is incorrectly classified as an anomaly, and FN arise when anomalies are mistakenly classified as normal. Together these provide a comprehensive evaluation of the proposed model's performance and its ability to accurately detect anomalies in the dataset.

An effective ADS should be capable of achieving a high F1-score while maintaining a low execution time. The F1-score is a useful metric as it considers both precision and recall, calculating their harmonic average to provide an overall assessment of the model's performance. Therefore, a high F1-score indicates that the ADS model has both high precision and high recall, which means it detects a significant proportion of anomalies while minimizing false positives. However, the execution time of the ADS, particularly the model training time is also crucial as it offers information about the computational performance. Therefore, achieving a balance between a high F1-score and low execution time is crucial for developing an effective ADS.

2) *Dataset*: To evaluate the proposed ADS model, we used the "CAN-intrusion dataset" provided by the Hacking and Countermeasure Research Lab (HCRL) [32]. This dataset, tailored for ADS/IDS development on the CAN-bus, was obtained by monitoring CAN traffic through a vehicle's OBD-II port. The intrusions in the dataset were created through message injection attacks, with each intrusion lasting between 3 to 5 seconds. The dataset, spanning 30-40 minutes of CAN traffic, features instances of impersonation attacks that occur sporadically without a predictable pattern, adding complexity to the detection process, and was processed by combining data and removing missing values for the development of the proposed ADS model. Table III outlines the details of the modified dataset.

3) *Performance Analysis*: On a system with an Intel(R) Core i7 – 8550 processor and 16 GB of RAM, the proposed model was trained and tested. Python programming language, PyCharm IDE version 2020.1, Keras, Pandas, and Xgboost libraries were utilized to develop the model, with Tenserflow as the backend.

The results of the proposed model on the CAN intrusion dataset are shown in Tables IV & V. As indicated by Table IV, the proposed model, which incorporates a blend of Rule-based and ML algorithms, demonstrates notable detection accuracy (above

90% in each instance) when evaluated on the CAN-intrusion dataset.

Comparing our proposed model to previous works on a similar dataset, it can be observed that although the approaches presented in the literature [33], [34] have achieved slightly better accuracies (ranging from 94% to 96%), they suffer from a comparatively high execution time in comparison to our proposed model shown in Fig. 3. This is attributed to the utilization of multi-threading enabled by DT, RF, XGBoost, CatBoost and LightGBM with the combination of Rule-based system in our proposed approach.

Therefore, the proposed model is well-suited for practical applications due to its high accuracy, F1-score, and low execution time (latency). In future works, it may be applied to real-time environmental data by taking into account the CAN-bus speed and CAN packet size.

V. RULE AND NEURAL NETWORKS BASED HYBRID ADS MODEL

Recently, there has been considerable admiration for Artificial Neural Networks (ANN), which are machine learning techniques widely used in applications involving large amounts of data. The construction of a neural network follows a similar pattern to how the human brain processes and makes decisions based on vital information. These neural networks were concocted to mimic the human brain's ability to excel in tasks such as recognizing images, patterns, etc., with high accuracy. The ability to train an ANN model enables substantial improvements in results.

Neural networks consist of a fundamental processing unit known as a neuron, similar to those found in the human brain, which transmits information from the brain to the nervous system. These neurons are interconnected through multiple layers and are responsible for processing information based on the weight allocated to each layer during the processing stage.

In a neural network, a neuron is designed to process multiple input features, x_i , each assigned with a corresponding weight, w_i . The neuron then performs computation to determine the weighted sum of these inputs. It is a standard approach to include a bias term b , in the calculation to enhance the neuron's output.

$$y = f \left(b + \sum_{i=1}^n x_i w_i \right) \quad (8)$$

The reason for selecting Deep Neural Network for ADS modeling lies in its capacity for training, which allows it to continually adapt and assimilate details about the regular and anomalous behavior of the system.

The rationales underpinning our decision to implement the DNN are:

- Neural Networks have low development costs and can be scaled more easily than other ML methods.
- The accuracy of statistical distribution provided by neural networks is high, and they make fewer assumptions. They can also be efficiently modified through learning.
- Also, they excel in minimizing false positives and false negatives, which are crucial factors in Anomaly Detection Systems.

TABLE IV
PERFORMANCE EVALUATION OF ADS ON CAN DATASET

Method	TP	FP	FN	TN	Precision	Recall (DR)	F1-Score	Accuracy
Rule-based (RB)	1,850,006	745,305	394,035	1,624,563	71.2%	82.4%	76.4%	75.29%
DT	2,159,016	61,616	85,025	2,307,174	97.3%	96.2%	96.1%	96.38%
RF	2,117,209	91,063	126,832	2,278,805	95.9%	94.3%	94.3%	94.71%
XGBoost	2,189,939	47,537	54,102	2,322,331	97.8%	97.5%	97.2%	97.62%
CatBoost	2,194,728	50,180	49,313	2,319,688	97.7%	97.8%	97.8%	97.83%
LightGBM	2,201,159	43,180	42,882	2,326,688	98.1%	98.1%	98.1%	98.05%
RB + DT	2,164,044	227,920	79,997	2,141,948	90.4%	96.4%	93.3%	93.25%
RB + RF	2,124,039	311,412	120,002	2,058,456	87.2%	94.6%	90.7%	90.63%
RB + XGBoost	2,194,164	183,047	49,877	2,186,821	92.3%	97.8%	94.8%	94.95%
RB + CatBoost	2,198,957	186,453	45,084	2,183,365	92.1%	97.9%	94.9%	94.97%
RB + LightGBM	2,203,909	187,260	40,132	2,182,608	92.1%	98.1%	95.05%	95.06%

TABLE V
EXECUTION TIME EVALUATION OF ADS ON CAN DATASET

Method	Execution Time (s)
KNN [33]	911.6
SVM [33]	13765.6
SAIDuCANT [34]	-
RB	132.006
DT	1278.4
RF	1905.3
XGBoost	2308.1
CatBoost	2076.7
LightGBM	1876.9
RB + DT	304
RB + RF	412.6
RB + XGBoost	350.21
RB + CatBoost	321.4
RB + LightGBM	297.15

Thus in this section, we present a hybrid ADS comprising two stages: a rule-based detection component and a Deep Neural Networks-based detection component (Model2), as depicted in Fig. 3.

The rule-based component offers a robust and prompt response by identifying malicious messages that overtly violate pre-defined rules, as discussed in Section IV. The neural networks component further enhances accuracy by identifying malicious messages that pass the rules in the first stage in a shorter time. Furthermore, the integration of the DNN component in the proposed hybrid ADS confers a high degree of versatility, making it capable of effectively detecting and classifying a broad spectrum of attacks that are otherwise challenging to handle using traditional machine learning (ML) techniques.

Specifically, we evaluated the model's performance on three different datasets and successfully detected the five most important types of attacks with high detection accuracy. Thus, the proposed hybrid model not only provides superior accuracy but also offers robustness against various types of attacks while minimizing computational time.

A. Binary Classification Neural Networks

A binary classification neural network is a computational model that includes an input layer, several hidden layers, and an output layer. The layers are composed of multiple neurons, and a single-node Sigmoid output layer for distinguishing between normal and attack classes.

They are used for binary classification tasks, where the goal is to classify an input into one of two possible classes, and hence there is only one output node that gives the probability of belonging to one of the two classes. In our case, they are trained to classify inputs into one of two categories - normal or attack.

In our model, the subsequent step after obtaining data from the initial stage 1 involves developing a binary classification neural network using the TensorFlow API.

The neural network is created through a sequential method by adding two dense layers with 40 and 20 neurons, respectively, along with input feature size. Only one neuron is in the output layer, as the output is either 1 (normal) or 0 (attack) for classification purposes.

Following the standard hyperparameters and optimization strategies of neural networks for binary classification [35], the activation function, 'Relu' is used at the input layer to ensure non-negative output from all neurons. At the output layer, the 'Sigmoid' activation function is employed to specify binary classification by determining the size of the output label. The model is trained using 'Adam' optimizer with a learning rate of 0.01 and is compiled with a binary cross-entropy loss function model, which measures the difference between the predicted probability of the positive class and the true probability of the positive class, i.e., it penalizes the network for making incorrect predictions and rewards it for making correct predictions. Table VI shows the hyper-parameter list of Binary Classification Neural Network.

Finally, the model is trained using xtrain (featuring values used for training and prediction) and ytrain (output label values), and the performance metrics are reckoned.

B. Multiclass Classification Neural Networks

A multiclass classification neural network is a computational model that includes an input layer, several hidden layers, and multiple output layers.

TABLE VI
THE HYPER-PARAMETERS OF BINARY CLASSIFICATION NEURAL NETWORK

Hyper-Parameters	Values
Number of Hidden Layers	2
Units in Hidden Layer	40, 20
Hidden layer activation	Relu
Output activation function	Sigmoid
Optimizer	Adam
Loss function model	Binary cross-entropy
Learning rate	0.01
Dropout	-

TABLE VII
THE HYPER-PARAMETERS OF MULTICLASS CLASSIFICATION NEURAL NETWORK

Hyper-Parameters	Values
Number of Hidden Layers	3
Units in Hidden Layer	40, 40, 20
Hidden layer activation	Relu
Output activation function	Softmax
Optimizer	Adam
Loss function model	Categorical cross-entropy
Learning rate	0.01
Dropout	0.2

They are used for classification tasks where there are more than two possible classes, and hence there are multiple output nodes, each corresponding to one of the possible classes, and the highest output value indicates the predicted class. In the context of our study, they are trained to categorize into diverse attack categories and eventually exhibit impressive accuracy levels during testing and uses multiple hidden layers and a Softmax-activated output layer for multi-category attack classification.

For our model, the next step after obtaining filtered data from the initial stage of the rule-based system involves developing a multiclass classification neural network using TensorFlow.

The neural network is created through a sequential method by adding three dense layers with 40, 40, and 20 neurons, respectively, along with input feature size. The number of neurons in the output layer of a multiclass classification neural network depends on the number of classes in the classification task. The values of these neurons represent the probability of the input belonging to the corresponding class. Table VII shows the hyper-parameter list of Multiclass Classification Neural Network.

C. Performance Evaluation

1) *Validation Metrics*: The assessment methodology and the validation metrics remain unchanged and are consistent with the procedure explicated in Section IV-E-1 of this paper.

2) *Datasets*: This study utilizes three distinct datasets to assess the performance of the proposed hybrid Anomaly Detection Systems (ADS), which combines rule-based and neural network-based (binary and multiclass) models. Table VIII

presents a compendium of the specifics concerning the datasets employed. These datasets are:

- **CAN INTRUSION DATASET**: As discussed in Section IV-E-2, the dataset we utilized was provided by HCRL specifically for developing ADS on CAN BUS. This dataset, sourced from a Kia Soul vehicle, captures both benign and malicious activities. While instances of DOS and Fuzzy attacks are periodic, our analysis during the data-preprocessing stage reveals that among Impersonation attacks, 18% are non-periodic, with the remainder being periodic. Notably, it is the only publicly available CAN dataset with remote frames and responses. The main attributes of this dataset encompass timestamp, CAN ID, DLC, and payload.
- **CAR HACKING DATASET**: This dataset [37] was created for experiments by connecting two Raspberry Pi devices to a vehicle's in-vehicle network via the OBD-II port, which allowed them to send and receive messages from real ECU nodes on the CAN-bus. The intrusions present in this dataset encompass three distinct types of attacks, specifically DOS attacks, Fuzzy attacks, and Spoofing attacks. While DOS and Fuzzy attacks are primarily periodic, Spoofing attacks have both periodic and non-periodic instances, with almost 26% of the instances being non-periodic, as observed during our data pre-processing. Notably, each dataset attribute—timestamp, CAN ID, DLC, payload, and label—serves as a crucial pointer, differentiating between the injected malicious messages and the normal ones. A significant volume of attack instances were recorded, leading to noticeable ID frequency alterations. Such marked changes indicate that sequence-based detection methodologies could be particularly effective in identifying these intrusions.
- **AUTOMOTIVE CAN BUS INTRUSION DATASET**: This dataset [38], was released by the Department of Mathematics and Computer Science at Eindhoven University of Technology. It offers a comprehensive collection of CAN bus data acquired from three distinct sources: two cars, the Opel Astra and Renault Clio, and a custom-built CAN-bus prototype. While benign data was collected directly from the vehicles, a variety of simulated attacks enriched the dataset, including Replay attacks, DOS attacks, and fuzzy attacks. The diversity of attack types and the non-periodic nature of some attacks make this dataset a valuable resource for developing and testing ADS.

3) *Performance Analysis*: The implemented hybrid model underwent a series of training and testing procedures on a computational system equipped with an Intel(R) Core i7-8550 as stated in Section IV-E-3. For the model, we utilized Python version 3, scikit-learn, while Tensorflow was designated as the underlying framework.

The results of the proposed model on the three different dataset are shown in Table IX & Table X. As shown through these tables, when testing on different datasets, the proposed model, which includes a combination of Rule-based and neural networks, achieves quite good accuracies (above 95% for both Binary and Multiclass).

If we compare these to the results in Section IV-E-3, the approach proposed here has achieved higher accuracies

TABLE VIII
DATASETS

Datasets	Number of Instances					
	DOS	Fuzzy	Impersonation	Spoofing	Replay	Attack Free State
CAN Intrusion Dataset	656,579	591,990	995,472	-	-	2,369,868
Car Hacking Dataset	3,665,771	3,838,860	-	4,621,702	-	988,987
Automotive CAN Bus Intrusion Dataset	9,22,970	9,22,970	-	-	9,22,970	30,76,636

TABLE IX
PERFORMANCE EVALUATION OF HYBRID RULE BASED AND BINARY CLASSIFICATION NEURAL NETWORK MODEL ON DIFFERENT DATASETS

Dataset	Attacks	TP	FP	FN	TN	Recall (DR) (%)	F1-Score (%)	Accuracy (%)	Execution Time (s)
CAN Intrusion Dataset	DOS	2,217,558	22,517	26,483	2,347,351	98.82	98.83	98.52	209.7
	Fuzzy								
	Impersonation								
Car Hacking Dataset	DOS	12,000,917	50,017	125,416	938,970	98.97	99.27	98.23	312.57
	Fuzzy								
	Spoofing								
Automotive CAN Bus Intrusion Dataset	DOS	2,713,947	72,689	32,963	3,003,947	98.80	97.47	97.73	276.08
	Fuzzy								
	Replay								

TABLE X
PERFORMANCE EVALUATION OF HYBRID RULE BASED AND MULTICLASS CLASSIFICATION NEURAL NETWORK MODEL ON DIFFERENT DATASETS

Dataset	Attacks	TP	FP	FN	TN	Recall (DR) (%)	F1-Score (%)	Accuracy (%)	Execution Time (s)
CAN Intrusion Dataset	DOS	632,348	29,993	24,231	2,339,875	96.30	96.22	96.23	315.76
	Fuzzy	565,282	28,947	26,708	2,340,921	95.47	95.47	95.72	
	Impersonation	959,211	32,014	36,261	2,337,854	96.35	96.26	96.38	
Car Hacking Dataset	DOS	3,521,120	42,031	144,651	946,956	96.07	96.07	96.07	488.04
	Fuzzy	3,728,127	41,047	110,733	947,940	97.10	96.98	96.89	
	Spoofing	4,493,215	45,039	128,487	943,948	97.22	97.22	97.22	
Automotive CAN Bus Intrusion Dataset	DOS	884,969	49,005	37,971	3,027,631	95.88	95.71	95.92	397.83
	Fuzzy	878,123	48,013	44,847	3,028,623	95.13	94.95	95.14	
	Replay	889,631	43,028	33,339	3,033,608	96.39	96.31	96.34	

(95%–98%) and relatively low execution time, attributed to the neural networks combined with the rule-based system. The model also shows significantly lower false negative rates, which is due to the neural network’s ability to learn and detect more complex attack patterns that might elude rule-based systems, thereby reducing missed anomalies. This balance of higher detection accuracy, reduced false negatives, and low execution time (latency) demonstrates the overall efficiency of our model.

To further benchmark our hybrid anomaly detection approach, we conducted a comparative evaluation against the LCCDE ensemble methodology proposed by Yang et al. [36], a decision-based ensemble learning framework for CAN Bus intrusion detection. Although our architecture differs in model structure and

learning paradigm, this comparison provides additional context for the relative performance of HAVEN.

We replicated the LCCDE-style ensemble using the same base learners (LightGBM, XGBoost, and CatBoost) and evaluated it on the three datasets used in our study. Table XI shows the detailed results for the Car Hacking Dataset, which offers the most direct point of comparison. While the LCCDE-style ensemble achieved slightly higher accuracy, HAVEN attained higher precision and F1-score, underscoring its more reliable classification behavior and lower false-positive tendency. For the CAN Intrusion and Automotive CAN Bus datasets, the LCCDE-style ensemble exhibited slightly modest accuracy improvements of around 0.3–0.5% over HAVEN, though the overall

TABLE XI
PERFORMANCE COMPARISON ON CAR HACKING DATASET

Metric	HAVEN (Table 9)	LCCDE (replicated)	LCCDE [36]
Accuracy (%)	98.23	98.60	99.9997
Precision (%)	99.58	99.12	99.9997
Recall (%)	98.97	99.01	99.9997
F1-Score (%)	99.27	99.05	99.9997
Exec. Time (sec)	312.57	290.41	185.10

precision and F1-score remained comparable or in favor of our approach. However, the overall gain remained incremental and did not match the near-perfect metrics reported in [36] likely due to variations in preprocessing, class balancing, or feature engineering steps not fully specified in the original study.

While the LCCDE ensemble showed a reduction in execution time compared to HAVEN, the difference is expected due to the additional complexity introduced by the neural network-based learning phase in our approach. This trade-off allows HAVEN to reduce false alarms while maintaining strong detection capability, enabling better modeling of temporal dependencies and feature semantics, particularly in multiclass or rule-assisted settings where static ensemble voting may underperform. Overall, HAVEN offers a competitive and balanced solution with high detection performance, adaptability and practical applicability across diverse CAN datasets.

VI. CONCLUSION

In this paper, we presented two novel two-step ADS's models for in-vehicle CAN buses. In first model, we employed a rule-based component to check injected CAN signals in the first step and tree-based machine learning methods in the second step. The SMOTE oversampling technique was used to lessen the impact of class imbalance and computational expense. The results on the CAN-intrusion dataset reveal that the proposed system provides an detection accuracy of above 90% with very low execution time (in all three cases), thus showing a better trade-off between the accuracy and the execution time. The second model employs rule-based discovery in combination with deep neural networks, including binary and multiclass classifiers which was something missing from previous works. The model was tested on three large datasets and achieved high detection accuracy rates of over 95% for all three datasets while also maintaining low execution times, including various types of attacks such as DOS, Fuzzy, Impersonation, Spoofing, and Replay attacks. The optimization of performance in the model is attained through the meticulous adjustment of specific parameters while testing and examining the model's functioning through fine-tuning parameters such as the number of hidden layers, the encoding methods employed, and the model's dropout and learning rate. After thorough evaluation, it is evident that our second hybrid ADS model integrating neural networks outperforms the tree-based machine learning variant in both accuracy and execution efficiency. This advantage stems from the neural network's ability to analyze complex, high-dimensional data and swiftly adapt to nuanced patterns. While tree-based models such as LightGBM, XGBoost, and CatBoost offer strong performance across accuracy, recall, and F1-scores, they do not consistently provide faster inference times. DL models, on the other hand, show superior performance in detecting non-periodic attacks

like spoofing and impersonation, while maintaining low execution latency. This highlights the complementary strengths of both approaches, validating our hybrid architecture that strategically integrates them for a balanced and robust anomaly detection system

As a part of our future endeavors and a natural progression of this research, we aim to augment the performance of our hybrid ADS model by continuously seeking and scrutinizing new rules and machine learning algorithms that could potentially provide superior results. Furthermore, we can enhance the proposed model's outcomes on datasets by optimizing the hyper-parameters of the algorithms, which can be achieved by utilizing advanced optimization techniques such as particle swarm and Bayesian optimization. This would aid in fine-tuning the model parameters and eventually lead to better results. Furthermore, we intend to incorporate emerging advancements in machine learning, including one-class classifiers, autoencoders, and self-supervised anomaly detection methods, to improve resilience against previously unseen attack types and reduce dependency on labeled data. Finally, evaluating the model in real-time conditions using live CAN traffic is another important direction that will help validate HAVEN's detection latency and practical deployment feasibility.

REFERENCES

- [1] B. Marilisa et al., "C-its communication: An insight on the current research activities in the European union," *Int. J. Transp. Syst.*, vol. 3, pp. 52–63, 2018.
- [2] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in *Proc. 16th Annu. Conf. Privacy, Secur. Trust*, 2018, pp. 1–6.
- [3] D. K. Nilsson, U. E. Larson, and E. Jonsson, "Efficient in-vehicle delayed data authentication based on compound message authentication codes," in *Proc. IEEE 68th Veh. Technol. Conf.*, 2008, pp. 1–5.
- [4] C. Szilagyi and P. Koopman, "Flexible multicast authentication for time-triggered embedded control network applications," in *Proc. 2009 IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2009, pp. 165–174.
- [5] L. B. Othmane, L. Dhulipala, M. Abdelkhalek, N. Multari, and M. Govindarasu, "On the performance of detecting injection of fabricated messages into the CAN bus," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 468–481, Jan./Feb. 2022.
- [6] H. Schweppe, Y. Roudier, B. Weyl, L. Apvrille, and D. Scheuermann, "Car2X communication: Securing the last meter—a cost-effective approach for ensuring trust in Car2X applications using in-vehicle symmetric cryptography," in *Proc. 2011 IEEE Veh. Technol. Conf.*, 2011, pp. 1–5.
- [7] A. Van Herreweghe, D. Singelee, and I. Verbauwhede, "CANAuth-A simple, backward compatible broadcast authentication protocol for CAN bus," in *Proc. ECRYPT Workshop Lightweight Cryptography*, 2011, vol. 2011, pp. 229–235. [Online]. Available: https://perso.uclouvain.be/fstandae/LC_2011_proceedings.pdf
- [8] J. Golson, "Car hackers demonstrate wireless attack on tesla model S," *Verge*, vol. 19, 2016. [Online]. Available: <https://www.theverge.com/2016/9/19/12985120/tesla-model-s-hack-vulnerability-keen-labs>
- [9] M. Muter and N. Asaj, "Entropy-based anomaly detection for in-vehicle networks," in *Proc. 2011 IEEE Intell. Veh. Symp.*, 2011, pp. 1110–1115.

- [10] M. Marchetti, D. Stabili, A. Guido, and M. Colajanni, "Evaluation of anomaly detection for in-vehicle networks through information-theoretic algorithms," in *Proc. IEEE 2nd Int. Forum Res. Technol. Soc. Ind. Leveraging Better Tomorrow*, 2016, pp. 1–6.
- [11] Z. Tyree, R. A. Bridges, F. L. Combs, and M. R. Moore, "Exploiting the shape of can data for in-vehicle intrusion detection," in *Proc. IEEE 88th Veh. Technol. Conf.*, 2018, pp. 1–5.
- [12] Y. Fraiji, L. B. Azzouz, W. Trojet, and L. A. Saidane, "Cyber security issues of internet of electric vehicles," in *Proc. 2018 IEEE Wireless Commun. Netw. Conf.*, 2018, pp. 1–6.
- [13] V. S. Barletta, D. Caivano, A. Nannavecchia, and M. Scalera, "A Kohonen SOM architecture for intrusion detection on in-vehicle communication networks," *Appl. Sci.*, vol. 10, no. 15, 2020, Art. no. 5062.
- [14] D. A. Schmidt, M. S. Khan, and B. T. Bennett, "Spline-based intrusion detection for VANET utilizing knot flow classification," *Internet Technol. Lett.*, vol. 3, no. 3, 2020, Art. no. e155.
- [15] D. Oladimeji, R. Jinad, A. Rasheed, and M. Baza, "CANGuard: An enhanced approach to the detection of anomalies in can-enabled vehicles," *Sensors*, vol. 25, no. 1, 2025, Art. no. 278.
- [16] A. Taylor, S. Leblanc, and N. Japkowicz, "Anomaly detection in automobile control network data with long short-term memory networks," in *Proc. 2016 IEEE Int. Conf. Data Sci. Adv. Analytics*, 2016, pp. 130–139.
- [17] A. Zhou, Z. Li, and Y. Shen, "Anomaly detection of CAN bus messages using a deep neural network for autonomous vehicles," *Appl. Sci.*, vol. 9, no. 15, 2019, Art. no. 3174.
- [18] N. Hamad, K. Alheeti, and S. Al-Rawi, "Intelligent intrusion detection system in internal communication systems for driverless cars," Ph.D. dissertation, Univ. Anbar, Ramadi, Iraq, 2020.
- [19] D. Kosmanos et al., "A novel intrusion detection system against spoofing attacks in connected electric vehicles," *Array*, vol. 5, 2020, Art. no. 100013. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S259000561930013X>
- [20] H. Olufowobi et al., "Anomaly detection approach using adaptive cumulative sum algorithm for controller area network," in *Proc. ACM Workshop Automot. Cybersecurity*, 2019, pp. 25–30.
- [21] H. M. Song, H. R. Kim, and H. K. Kim, "Intrusion detection system based on the analysis of time intervals of CAN messages for in-vehicle network," in *Proc. 2016 Int. Conf. Inf. Netw.*, 2016, pp. 63–68.
- [22] M. Mütter, A. Groll, and F. C. Freiling, "A structured approach to anomaly detection for in-vehicle networks," in *Proc. 6th Int. Conf. Inf. Assurance Secur.*, 2010, pp. 92–98.
- [23] G. Loukas, T. Vuong, R. Heartfield, G. Sakellari, Y. Yoon, and D. Gan, "Cloud-based cyber-physical intrusion detection for vehicles using deep learning," *IEEE Access*, vol. 6, pp. 3491–3508, xbrk 2018.
- [24] M. Aloqaily, S. Otoum, I. Al Ridhawi, and Y. Jararweh, "An intrusion detection system for connected vehicles in smart cities," *Ad Hoc Netw.*, vol. 90, 2019, Art. no. 101842.
- [25] K.-T. Cho and K. G. Shin, "Fingerprinting electronic control units for vehicle intrusion detection," in *Proc. USENIX Secur. Symp.*, 2016, vol. 40, pp. 911–27.
- [26] S. Purohit and M. Govindarasu, "ML-based anomaly detection for intra-vehicular CAN-bus networks," in *Proc. 2022 IEEE Int. Conf. Cyber Secur. Resilience*, 2022, pp. 233–238.
- [27] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
- [28] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *J. Mach. Learn. Res.*, vol. 3, no. 3, pp. 1157–1182, 2003.
- [29] D. Zhang, L. Qian, B. Mao, C. Huang, B. Huang, and Y. Si, "A data-driven design for fault detection of wind turbines using random forests and XGboost," *IEEE Access*, vol. 6, pp. 21020–2103, 2018.
- [30] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, vol. 31. [Online]. Available: https://papers.nips.cc/paper_files/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html
- [31] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html
- [32] "Can-intrusion-dataset (OTIDS)," (n.d.). [Online]. Available: <http://ocslab.hksecurity.net/Dataset/CAN-intrusion-dataset>
- [33] A. Alshammari, M. A. Zohdy, D. Debnath, and G. Corser, "Classification approach for intrusion detection in vehicle systems," *Wireless Eng. Technol.*, vol. 9, no. 4, pp. 79–94, 2018.
- [34] H. Olufowobi, C. Young, J. Zambreno, and G. Bloom, "SAIDuCANT: Specification-based automotive intrusion detection using controller area network (CAN) timing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 1484–1494, Feb. 2020.
- [35] A. Géron, *Hands-On Machine Learning With Scikit-Learn, Keras, and TensorFlow*. Sebastopol, CA, USA: O'Reilly Media, Inc., 2022.
- [36] L. Yang, A. Shami, G. Stevens, and S. De Rusett, "LCCDE: A decision-based ensemble framework for intrusion detection in the Internet of Vehicles," in *Proc. 2022 IEEE Glob. Commun. Conf.*, 2022, pp. 3545–3550.
- [37] "Car-hacking dataset," (n.d.). [Online]. Available: <https://ocslab.hksecurity.net/Datasets/car-hacking-dataset>
- [38] G. Dupont, A. Lekidis, J. Den Hartog, and S. Etalle, "Automotive controller area network (CAN) bus intrusion dataset V2." 4TU. Centre for Research Data, Nov. 14, 2019, doi: [10.4121/UID: B74B4928-C377-4585-9432-2004DFA20A5D](https://doi.org/10.4121/UID: B74B4928-C377-4585-9432-2004DFA20A5D).



such as autonomous vehicles, EV charging infrastructure, and distributed energy resources to enhance their security and reliability. He was honored with the Research Excellence Award.



Manimaran Govindarasu (Fellow, IEEE) is currently the Anson Marston distinguished professor of engineering and Murray J. and Ruth M. Harpole professor of electrical and computer engineering with Iowa State University. He has co-authored nearly 250 peer-reviewed research publications. His research interests include cybersecurity for smart grid and critical infrastructures, IoT security, internet security, and real-time systems. He was the recipient of the multiple best paper awards, presented several dozen invited talks, tutorials, and industry short-courses/hands-on training sessions on cybersecurity. He was an associate editor for *IEEE Transactions on Smart Grid* and *IEEE Transactions of Mobile Computing*. He is the chair of the Cybersecurity Working Group in IEEE Power and Energy Society. He led the effort to build ISU's Cybersecurity testbed for smart grid and successfully collaborated with many universities, national laboratories, and industries in R&D projects. His research program received funding from NSF, DOE, DHS, DoD, PSERC, and ISU EPRC. He is an ABET Program Evaluator.